

Reviewer Pack — Minimal Order of Arithmetic Cycles and Communication Complex...

Entry b69aa406f2e0 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-02 16:48:31 UTC

Minimal Order of Arithmetic Cycles and Communication Complexity Rank Correlation

Entry ID: b69aa406f2e0 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-02 16:48:31 UTC

1. Conjecture

Field A (mathematical branch): Arithmetic Geometry (specifically, number theory of cycles) **Field B** (complexity object): Communication Complexity

Statement:

For every k -CNF φ with n variables, the minimal order of arithmetic cycles in its associated algebraic variety is linearly correlated with its communication complexity rank $r(\varphi)$, such that $O(n^2 \log n) \leq m_{\text{order}}(\varphi) \leq C r(\varphi)$ for some constant C .

Rationale (proposer's reasoning):

Arithmetic cycles provide a novel geometric invariant that could potentially expose the underlying structure of the CNF's algebraic variety. If this correlation holds, it would suggest that the arithmetic cycle structure can be leveraged to understand the complexity of communication protocols.

Taxonomy category: communication_complexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): ff158fe53045bb96

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if and only if, for all k -CNFs φ with n variables ($n \leq 40$), the correlation between $m_{\text{order}}(\varphi)$ and $r(\varphi)$ meets the following conditions: $O(n^2 \log n) \leq m_{\text{order}}(\varphi) \leq C r(\varphi)$ where $\text{support_fraction} = \text{number_of_CNFs_meeting_conditions} / \text{total_number_of_CNFs} \geq 0.8$ AND $\text{metric_mean of } m_{\text{order}}(\varphi) \text{ across all CNFs} \leq C$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	0.95	UNCERTAIN	HITS
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 2 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "arithmetic geometry" AND "communication complexity" AND minimal order - "CNF" AND arithmetic cycles AND communication complexity rank - "algebraic variety" AND number theory AND communication complexity

Top relevant hits considered: - [s2:2502.10590] Optimal Bounds for the Number of Pieces of Near-Circuit Hypersurfaces - [s2:1608.02198] A General Characterization of the Statistical Query Complexity

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.3s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_k_cnf(n, k):
        clauses = []
        for _ in range(k):
            clause = [random.randint(1, n), -random.randint(1, n)]
            clauses.append(clause)
        return clauses

    def compute_algebraic_variety(cnf):
        # Placeholder function to simulate algebraic variety computation
        return len(cnf) # Simplified for demonstration purposes

    def minimal_order_of_arithmetic_cycles(variety):
        # Placeholder function to simulate arithmetic cycle computation
        return variety * variety # Simplified for demonstration purposes

    def communication_complexity_rank(cnf):
        # Placeholder function to simulate communication complexity rank computation
        return len(cnf) # Simplified for demonstration purposes

    n = random.randint(5, 40)
    k = random.randint(1, n // 2)
    cnf = generate_k_cnf(n, k)
    variety = compute_algebraic_variety(cnf)
    m_order = minimal_order_of_arithmetic_cycles(variety)
```

```

r_phi = communication_complexity_rank(cnf)

if m_order < n**2 * math.log(n) or m_order > 10 * r_phi:
    return {
        "metric_name": "m_order",
        "metric_value": m_order,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

    return {
        "metric_name": "m_order",
        "metric_value": m_order,
        "instances_tested": 1,
        "n_max": n,
        "conjecture_holds": True,
        "counterexample": ""
    }

if __name__ == "__main__":
    seeds = [int(x) for x in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3
    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    metric_values = [r["metric_value"] for r in results if r["conjecture_holds"]]
    support_fraction = len(metric_values) / len(results)
    metric_mean = sum(metric_values) / len(metric_values)

    if support_fraction >= 0.8 and metric_mean <= 10:
        print(f"RESULT: SUPPORTED mean={metric_mean} std=<y> support_fraction={support_fraction}")
    elif any(not r["conjecture_holds"] for r in results):
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample='mapping_undefined' first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE reason=insufficient_data")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

{'metric_name': 'm_order', 'metric_value': 64, 'instances_tested': 1, 'n_max': 18, 'conjecture_holds': True}
TRIAL: {'metric_name': 'm_order', 'metric_value': 4, 'instances_tested': 1, 'n_max': 15, 'conjecture_holds': True}
TRIAL: {'metric_name': 'm_order', 'metric_value': 1, 'instances_tested': 1, 'n_max': 7, 'conjecture_holds': True}
TRIAL: {'metric_name': 'm_order', 'metric_value': 36, 'instances_tested': 1, 'n_max': 25, 'conjecture_holds': True}
TRIAL: {'metric_name': 'm_order', 'metric_value': 36, 'instances_tested': 1, 'n_max': 23, 'conjecture_holds': True}
TRIAL: {'metric_name': 'm_order', 'metric_value': 1, 'instances_tested': 1, 'n_max': 9, 'conjecture_holds': True}
TRIAL: {'metric_name': 'm_order', 'metric_value': 36, 'instances_tested': 1, 'n_max': 38, 'conjecture_holds': True}
TRIAL: {'metric_name': 'm_order', 'metric_value': 64, 'instances_tested': 1, 'n_max': 16, 'conjecture_holds': True}

```

```

TRIAL: {'metric_name': 'm_order', 'metric_value': 100, 'instances_tested': 1, 'n_max': 36, 'conjecture': 'm_order=100'}
TRIAL: {'metric_name': 'm_order', 'metric_value': 49, 'instances_tested': 1, 'n_max': 19, 'conjecture': 'm_order=49'}
TRIAL: {'metric_name': 'm_order', 'metric_value': 4, 'instances_tested': 1, 'n_max': 17, 'conjecture': 'm_order=4'}

```

--STDERR--

Traceback (most recent call last):

File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_ec4f335d.py", line 77, in <module>

```
metric_mean = sum(metric_values) / len(metric_values)
```

```
~~~~~^~~~~~
```

ZeroDivisionError: division by zero

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents a proper evaluation of the conjecture. | next: Investigate and fix the crash in the test code to proceed with the verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	15975
2	preregistration	ollama_remote	glm4:latest	0	0	9927
3	novelty	ollama_remote	glm4:latest	0	0	11665
4	novelty	ollama_remote	glm4:latest	0	0	13462
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13386
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	20544
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	8114
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15565
9	judge	ollama_remote	glm4:latest	0	0	9284

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 117921 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in `research/audit/b69aa406f2e0.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/b69aa406f2e0.jsonl` for verification of provenance

4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: research/pvsnp_sandbox/test_*.py (per-cycle)

Replay tarball: research/replay/b69aa406f2e0.tar.gz (if generated)